

GCP Landing Zone: Phase 2 Network Provisioning Execution Plan

This document outlines the step-by-step tactical execution plan for Phase 2 of your GCP Landing Zone rollout: the **Network Backbone Provisioning Repository**. It details how the pipeline ingests the upstream `infra_map.json` artifact to construct isolated Shared VPC architectures and output an enriched `network_map.json` map for downstream workloads.

1. Core Pipeline Blueprint & State Lifecycle

Phase 2 operates as a completely decoupled infrastructure track to maximize platform stability and security boundaries. The operational pipeline flow follows a strict three-step execution framework:

```
``'+-----+ +-----+ +-----+ | 1. INGEST
ARCHITECTURE | ---> | 2. RUNTIME PROVISIONING | ---> | 3. OUTPUT HANDOFF MAP | |
Read values dynamically | | Create Host Projects, | | Compile & distribute | | from
'infra_map.json' | | Shared VPCs, & Subnets | | new 'network_map.json' | +-----+
+-----+ +-----+ +-----+ ```
```

A. Input Processing (Ingestion)

- The Network repository workflow references the `infra_map.json` compiled by your Foundation layer.
- It extracts the dynamic `project_parent` Folder IDs corresponding to your dev-infra, non-prod-infra, and prod-infra folder structures.

B. Output Processing (Distribution)

- Upon successful compilation of network assets, the pipeline aggregates newly generated deployment data.
- It exports a language-agnostic **`network_map.json`** blueprint mapping project IDs, VPC names, and subnet `self_links`.

2. Network Backbone Directory Tree Architecture

To maintain a modular, clean, and testable code structure, structure your new Network repository following this production layout:

```
gcp-network-backbone/
├── .github/
│   └── workflows/
│       └── network-deploy.yml           # Core GHA deployment pipeline
```

```

using WIF
|   └─ network-change-notifier.yml # Trigger script to alert
teams on JSON updates
└─ modules/
|   └─ network_host/
|       └─ main.tf # Provisions project, enables
Shared VPC host flag
|   └─ variables.tf
|       └─ outputs.tf
|   └─ vpc_infrastructure/
|       └─ main.tf # Provisions VPC, Subnets,
Routes, Cloud NAT
|   └─ variables.tf
|       └─ outputs.tf
└─ templates/
|   └─ network_map.tmpl # JSON heredoc template file
for handoff generation
└─ main.tf # Core composition calling
modules per environment
└─ variables.tf # Instantiates path pointers
to 'infra_map.json'
└─ outputs.tf # Holds final console
configuration blocks
└─ terraform.tfvars # Baseline region and
environmental string declarations
└─ infra_map.json # Upstream artifact brought in
at execution runtime

```

3. Core Technical Implementations

A. Parsing the Ingested Map

Inside the root execution layer, load the foundation file cleanly using native HCL functions without creating rigid string mappings:

```

locals {
  infra_data = jsondecode(file("${path.module}/infra_map.json"))

  # Dynamic folder extractions from upstream outputs
  dev_folder_id = local.infra_data.folders.dev_infra
  nonprod_folder = local.infra_data.folders.non_prod_infra
  prod_folder_id = local.infra_data.folders.prod_infra
}

```

B. Mandatory Network Host Project Configuration

Every target runtime environment requires its own isolated Network Host project. Ensure the network module enforces the host project declaration to transform standard projects into network providers:

```
# modules/network_host/main.tf
resource "google_project" "network_host" {
  name           = "net-host-${var.environment}"
  project_id     =
"company-net-${var.environment}-${var.random_suffix}"
  folder_id      = var.target_folder_id
  billing_account = var.billing_account_id
}

resource "google_compute_shared_vpc_host_project" "host_trigger" {
  project      = google_project.network_host.project_id
  depends_on   = [google_project.network_host]
}
```

4. Enriched Handoff Blueprint Layout (`network\_map.json`)

The end product of this layer is the compiled JSON metadata matrix. Downstream applications will read this structure locally via Strategy B to execute their Project Factory configurations:

```
{
  "environments": {
    "dev": {
      "host_project_id": "company-net-dev-1122",
      "vpc_name": "dev-shared-vpc",
      "subnets": {
        "app_tier":
"projects/company-net-dev-1122/regions/us-central1/subnetworks/dev-app-subnet",
        "db_tier":
"projects/company-net-dev-1122/regions/us-central1/subnetworks/dev-db-subnet"
      }
    },
    "non-prod": {
      "host_project_id": "company-net-nonprod-3344",
      "vpc_name": "nonprod-shared-vpc",
      "subnets": {
        "app_tier":
"projects/company-net-nonprod-3344/regions/us-central1/subnetworks/nonprod-app-subnet",
        "db_tier":
"projects/company-net-nonprod-3344/regions/us-central1/subnetworks/non"
```

```

prod-db-subnet"
    }
  },
  "prod": {
    "host_project_id": "company-net-prod-5566",
    "vpc_name": "prod-shared-vpc",
    "subnets": {
      "app_tier":
"projects/company-net-prod-5566/regions/us-central1/subnetworks/prod-app-subnet",
      "db_tier":
"projects/company-net-prod-5566/regions/us-central1/subnetworks/prod-db-subnet"
    }
  }
}

```

5. Automated Change Notifier Workflow (Slack / MS Teams)

To maintain strict visibility boundaries across product verticals, the Network repository implements an automated drift warning workflow. Whenever an administrator changes infrastructure details inside `network_map.json`, the pipeline alerts the engineering group:

- **Trigger Rule:** Enforces execution strictly when changes are merged into main targeting the JSON footprint exclusively (via GitHub paths scoping filters).
- **Guardrail Control:** Enforces a mandatory CODEOWNERS locking mechanism requiring signatures from platform engineering leads before any production network configuration maps can be modified.